Low-Level Design (LLD) – Chess Tournament Player Management System

Difficulty Level: Easy
Technology: Python OOPs
Total Marks: 10
Standards Followed: 4 Methods | 5 Test Cases

Problem Statement

A chess academy wants to manage players participating in a tournament. Each player has a unique Player ID, player name, rating, and participation status.

Design a class-based Python system that can:

- **Add a new player**
- **Update a player's chess rating**
- **Get player details**
- **List qualified players based on rating**

Store player information in a dictionary as shown below:

```
{
  "P101": {
    "name": "Arjun",
    "rating": 1850,
    "status": "Active"
  }
}
```

Class Declaration

class ChessTournamentSystem:

Operation 1: Add Player

Adds a new player to the tournament.

Function Prototype

def add_player(self, player_id: str, name: str, rating: int) -> dict:

Example Input

chess = ChessTournamentSystem()

chess.add_player("P101", "Arjun", 1850)

Expected Output

```
{
  "P101": {
    "name": "Arjun",
    "rating": 1850,
    "status": "Active"
  }
}
```

Implementation Flow

- **Check whether player_id already exists.**
- **If yes, raise:**

ValueError("Player already exists")

- **Add player with:**
  - **name**
  - **rating**
  - **status = "Active"**
- **Return updated dictionary.**

Operation 2: Update Rating

Updates the chess rating of an existing player.

Function Prototype

def update_rating(self, player_id: str, new_rating: int) -> dict:

Example Input

chess.update_rating("P101", 1920)

Expected Output

```
{
  "P101": {
    "name": "Arjun",
    "rating": 1920,
    "status": "Active"
  }
}
```

Implementation Flow

- **Check whether player exists.**
- **If not, raise:**

KeyError("Player not found")

- **Update rating.**
- **Return updated dictionary.**

Operation 3: Get Player Details

Returns details of a specific player.

Function Prototype

def get_player_details(self, player_id: str) -> dict:

Example Input

chess.get_player_details("P101")

Expected Output

```
{
  "name": "Arjun",
  "rating": 1920,
```

"status": "Active"

}

Implementation Flow

- **Check whether player exists.**
- **If not, raise:**

KeyError("Player not found")

- **Return player information.**

Operation 4: List Qualified Players

Returns all player IDs whose rating is greater than or equal to a given rating.

Function Prototype

def qualified_players(self, minimum_rating: int) -> list:

Example Input

chess.qualified_players(1800)

Expected Output

["P101", "P103"]

Implementation Flow

- **Iterate through all players.**
- **Select players whose:**

rating >= minimum_rating

- **Return matching player IDs as a list.**

Test Cases and Marks Allocation (10 Marks)

Test Case ID	Method	Validation Focus	Marks
TC1	add_player()	Adds player successfully	2
TC2	update_rating()	Updates rating correctly	2

TC3	**get_player_details()**	**Returns player information**	**2**
TC4	**qualified_players()**	**Returns players meeting rating criteria**	**2**
TC5	**Hidden**	**Handles duplicate player and missing player errors correctly**	**2**

Sample Input

chess.add_player("P101", "Arjun", 1850)

chess.add_player("P102", "Riya", 1700)

chess.add_player("P103", "Karthik", 2100)

chess.qualified_players(1800)

Sample Output

["P101", "P103"]